Layered Enterprise Architecture

Ehsan Khamespanah

Spring 2023

Layered Architecture

- Layering is a common practice to separate and organize code units by their role/responsibilities in the system.
- The main idea behind Layered Architecture is a separation of concerns: avoid mixing domain or database code with the UI stuff
- Layered Architecture gives us nothing apart from a guideline on how to organize the source code.



What Are Layers?

- A division of software into units (layers)
- Each layer is a virtual machine
  - Enhancing portability
- Constraints on the relationships between layers
- Tier vs. Layer
  - The two terms are often used as synonyms
  - Tier implies a physical separation
  - We don't have to run layers on different machines

Benefits of Layered Architecture

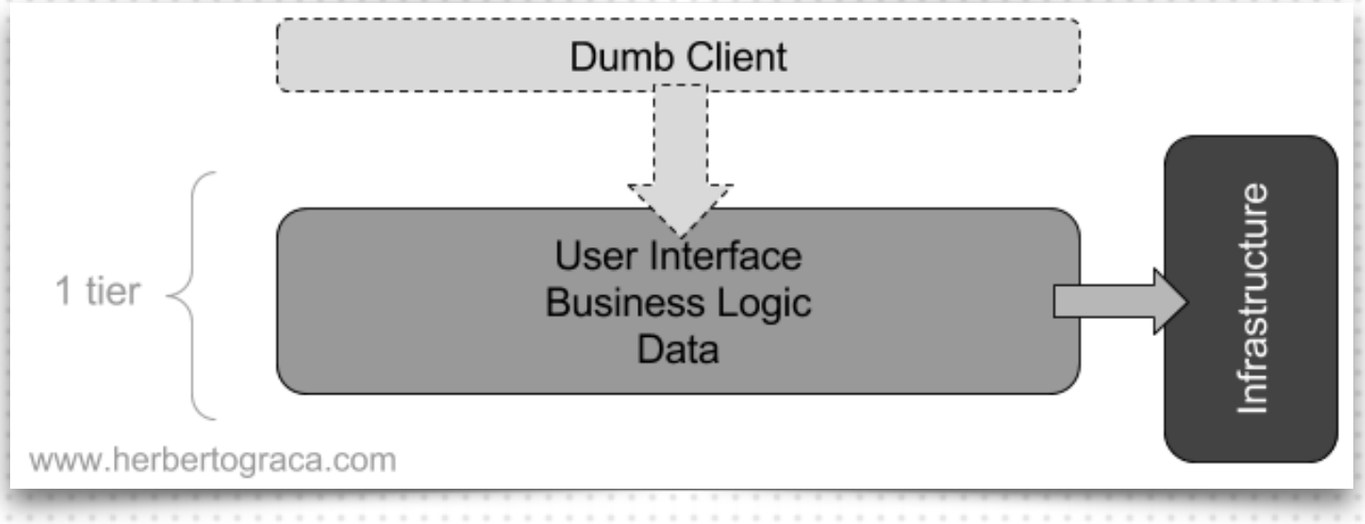
- Simplicity - Improving the Understandability
- Consistent Across Different Projects
- Guaranteed Separation of Concerns
- Alternative Implementation
- Minimising Dependency
- Standardisation

Drawbacks of a Layered Architecture

- Hidden Use Cases – you can't really say what a project is doing by simply looking at the code organization
- Low cohesion: classes that contribute to common scenarios and concepts are far from each other
- Not all things are encapsulated - Adding a field which should be presented in UI
- Extra layers harms applications performance
- Breaking OO Rules: Interface of layers are defined in terms of data

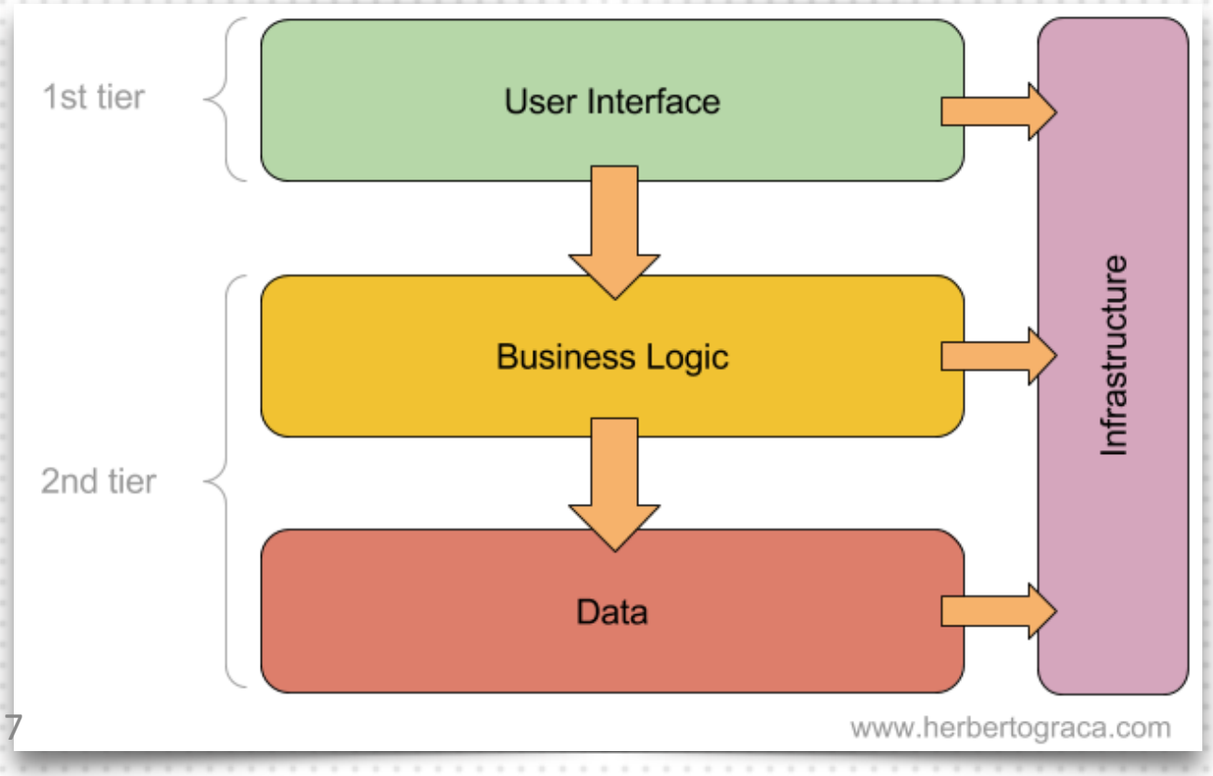
The 60s and 70s

- There was no GUI, all applications were usable only through a CLI (dumb terminals)
- Applications were quite simple
  - Weren’t built with layering in mind
  - They were deployed and used on one computer
- One-Tier applications
- Updating the software to a new version has to be done on every computer



Layering during the 80s and 90s

- Enterprise applications come to life
  - Several users in a company using desktop computers
- At this time, there were mostly three layers:
  - User Interface (Presentation)
  - Business Logic (Domain)
  - Data Source
- Two-Tier Applications

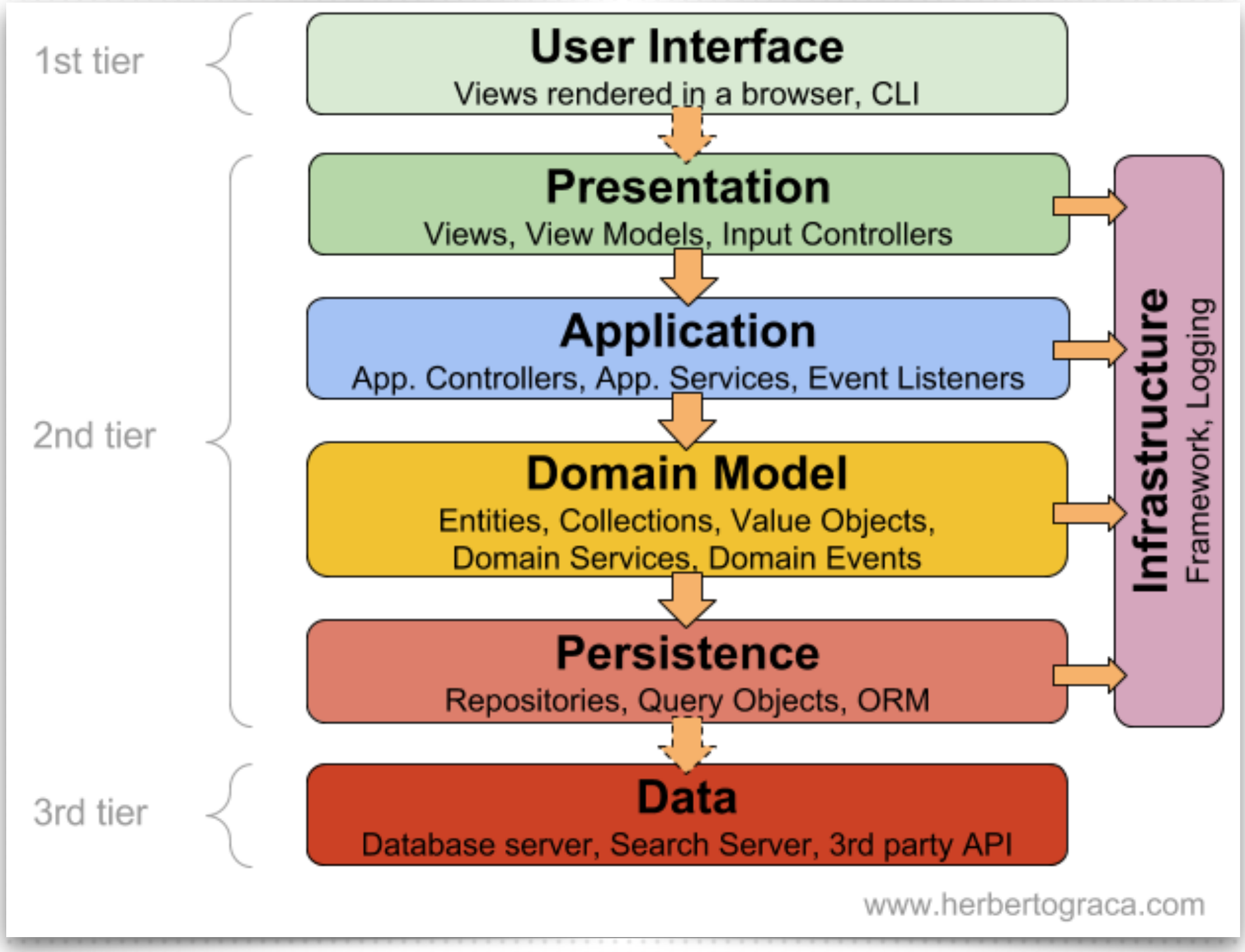


Layering during the 80s and 90s

- This architecture pattern solves the scalability problem
  - Several users could use the application independently
  - One-Tier application suffers from scalability problem
- Having a few hundred (even a few tenth of clients), updating the application would be a highly complex operation

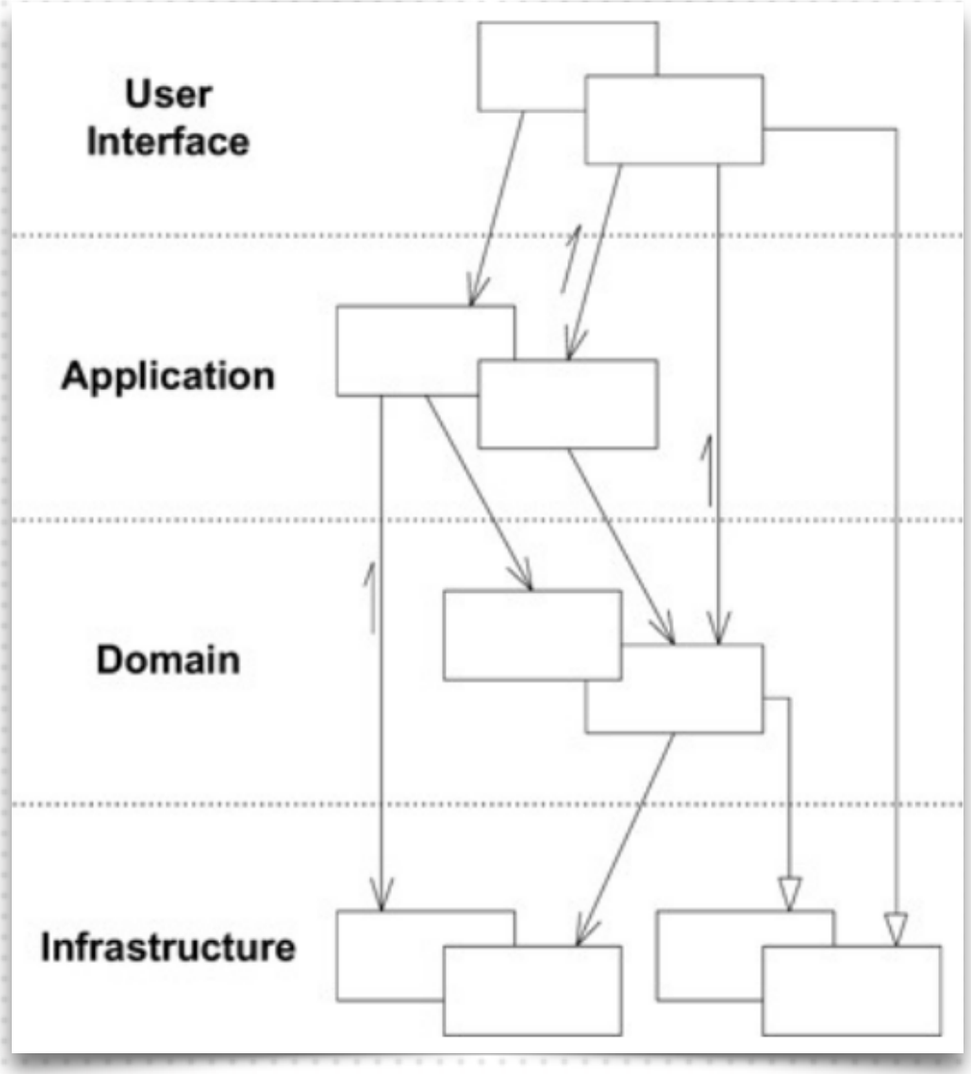
Layering after the mid 90s

- Shift to a cloud context, the increase in application users, application complexity and infrastructure complexity
  - **A native browser application**, rendering and running the user interface, sending requests to the server application;
  - **An application server**, containing the presentation layer, the application layer, the domain layer, and the persistence layer;
  - **A database server**, which would be used by the application server for the persistence of data.
- Three-tier architecture pattern, also known as n-tier
- It Solves the problem of updating the clients


1st tier
User Interface
Views rendered in a browser, CLI
2nd tier
Presentation
Views, View Models, Input Controllers
Application
App. Controllers, App. Services, Event Listeners
Domain Model
Entities, Collections, Value Objects,
Domain Services, Domain Events
Persistence
Repositories, Query Objects, ORM
3rd tier
Data
Database server, Search Server, 3rd party API
Infrastructure
Framework, Logging
www.herbertograca.com

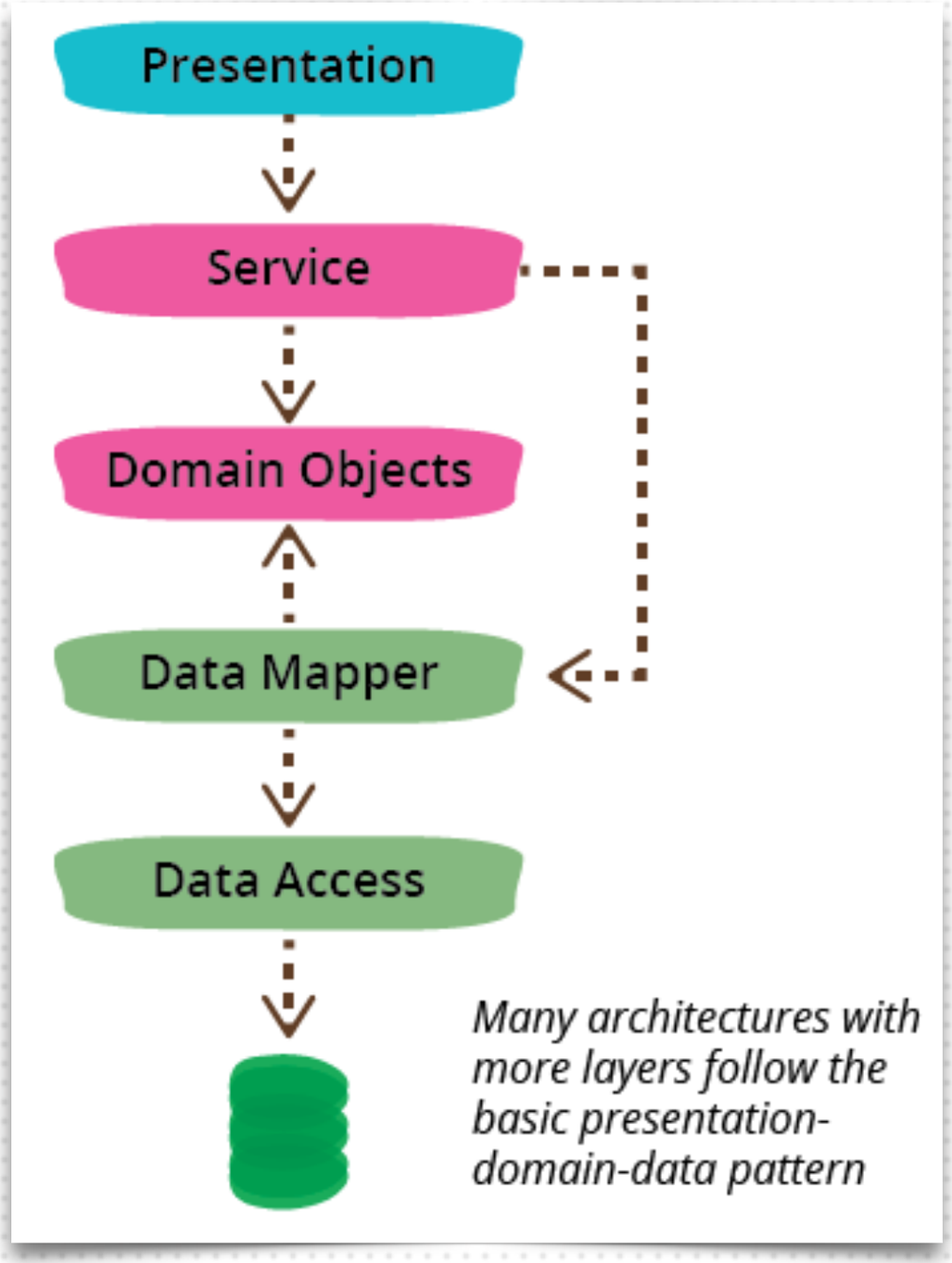
Layering after the early 2000s

- Domain-Driven Design: Tackling Complexity in the Heart of Software - Eric Evans, 2003
- A vision for the layering of a software system:
  - User Interface
  - Application Domain: EBIArchitecture
  - Domain Layer: The Heart of System
  - Infrastructure



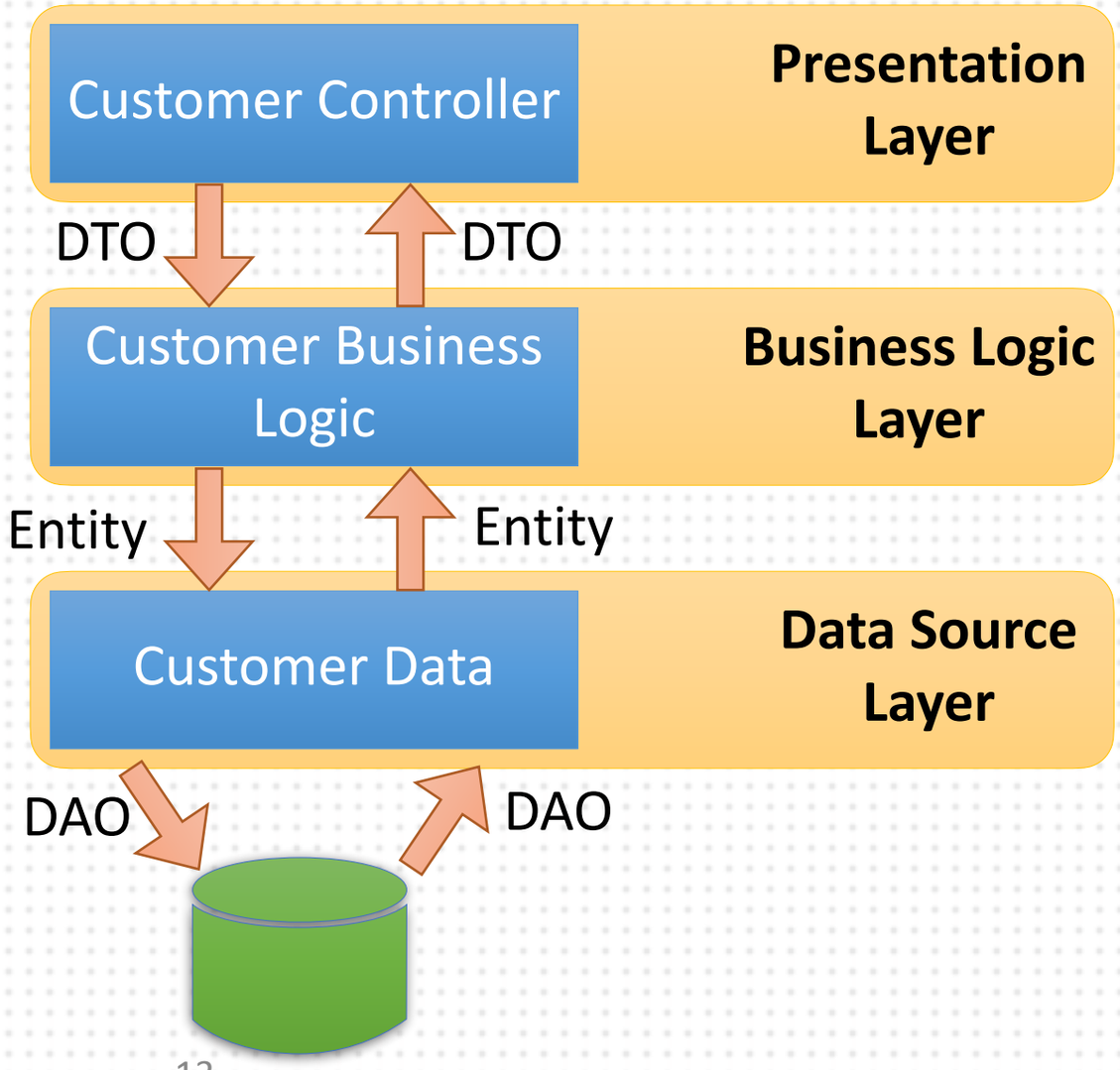
Being More Flexible

- Access from presentation model to data is not a layers breaks
  - Dependency from top to bottom
  - Data Mapper depends on both top and bottom layers

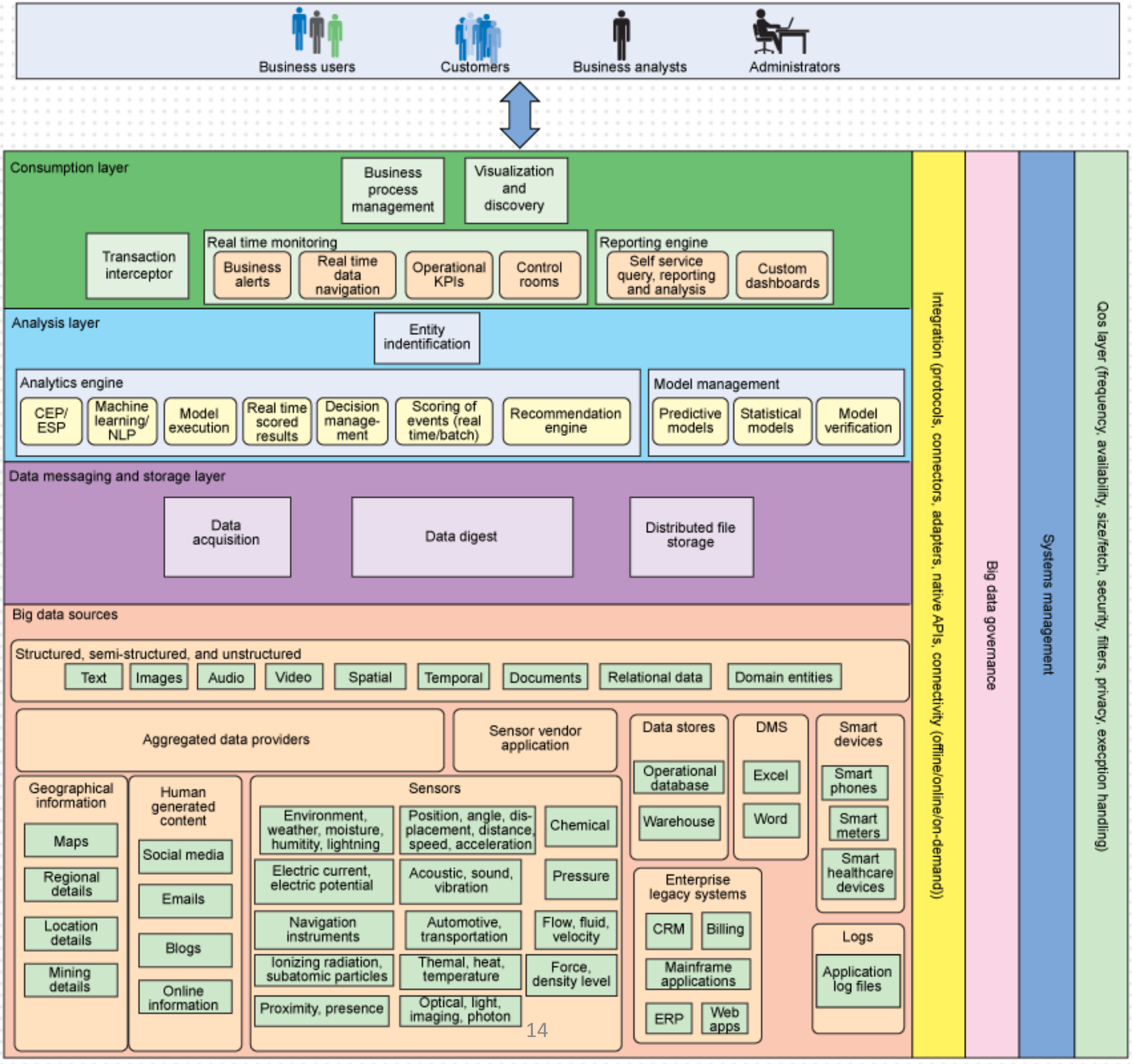


Flow of Data Among Layers

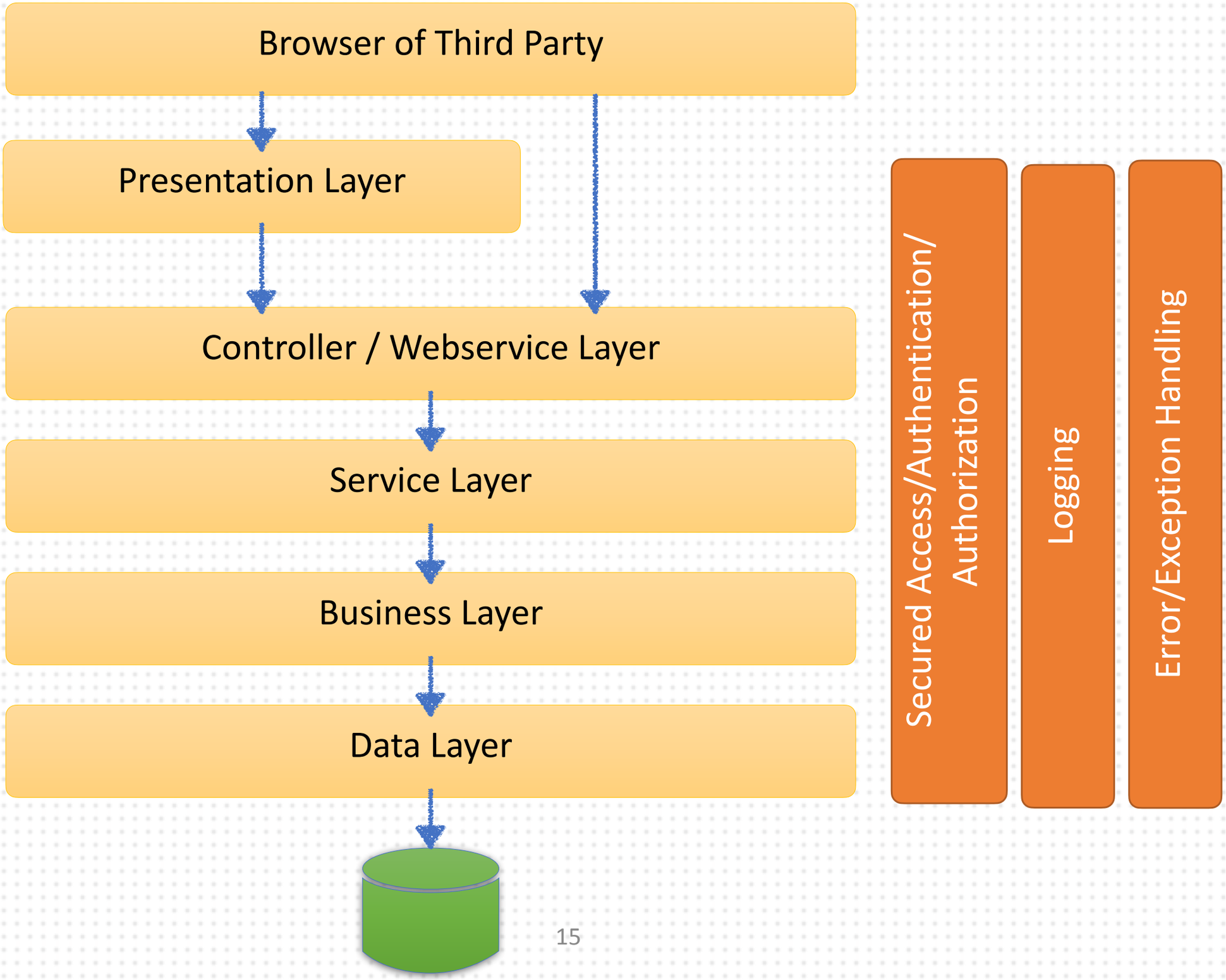
- Data Transfer Objects (DTO)
- Data Access Objects (DAO)
- DAO and Entity



Logical Layers of A Big Data Solution

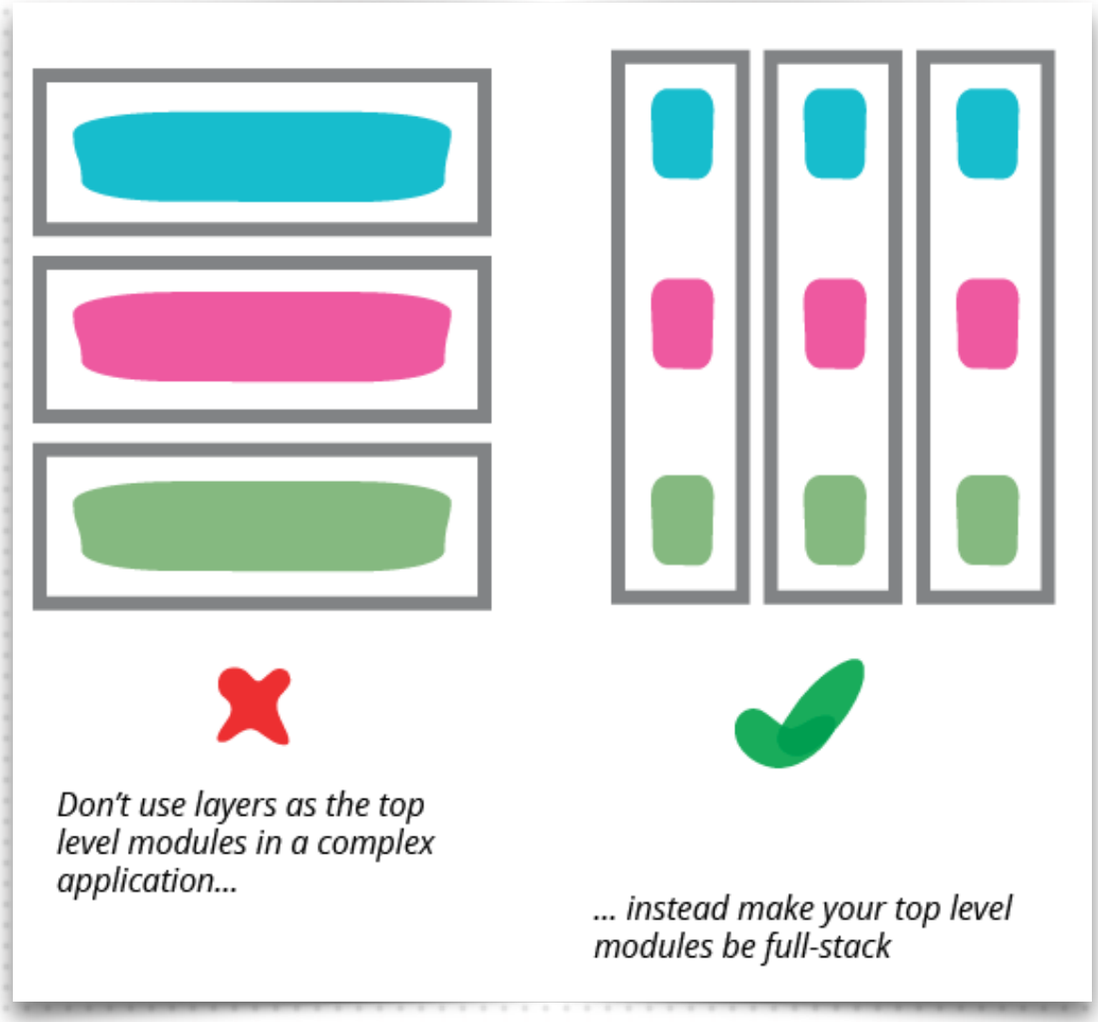


A Typical Case!



Attention!

- You should split your top level into domain oriented modules which are internally layered
- Don’t separate development teams by these layers
  - Developers don't have to be full-stack but teams should be.



Anti-pattern: Lasagna Architecture

- It happens when:
  - We decide to use a strict layering approach: ends up creating proxy methods, and even proxy classes
  - We lead the project into over-abstraction: Small updates reverberate through all areas of an application
  - Too many layers: increases the complexity of the overall system



Anti-pattern: Lasagna Architecture

- Too many tiers: increases the complexity and damages the performance of the overall system
- Organizing our monolith according to layers instead of sub-domains/components (ie. Product, Payment, Checkout)

Analysis of the Layered EA

- Overall agility is Low: is suffers from the monolithic nature of most implementations as well as the tight coupling of components
- Ease of deployment is Low: One small change to a component can require a redeployment of the entire application
- Testability is High: layers can be mocked or stubbed, making this pattern is relatively easy to test
- Performance is Low: the inefficiencies of having to go through multiple layers

Analysis of the Layered EA

- Scalability is Low: scaling a layered architecture by splitting the layers into separate physical deployments or replicating the entire application into multiple nodes
  - overall the granularity is too broad, making it expensive to scale
- Ease of development is High: most companies develop applications by separating skill sets by layers


BACK-END
FRONT-END
APIs